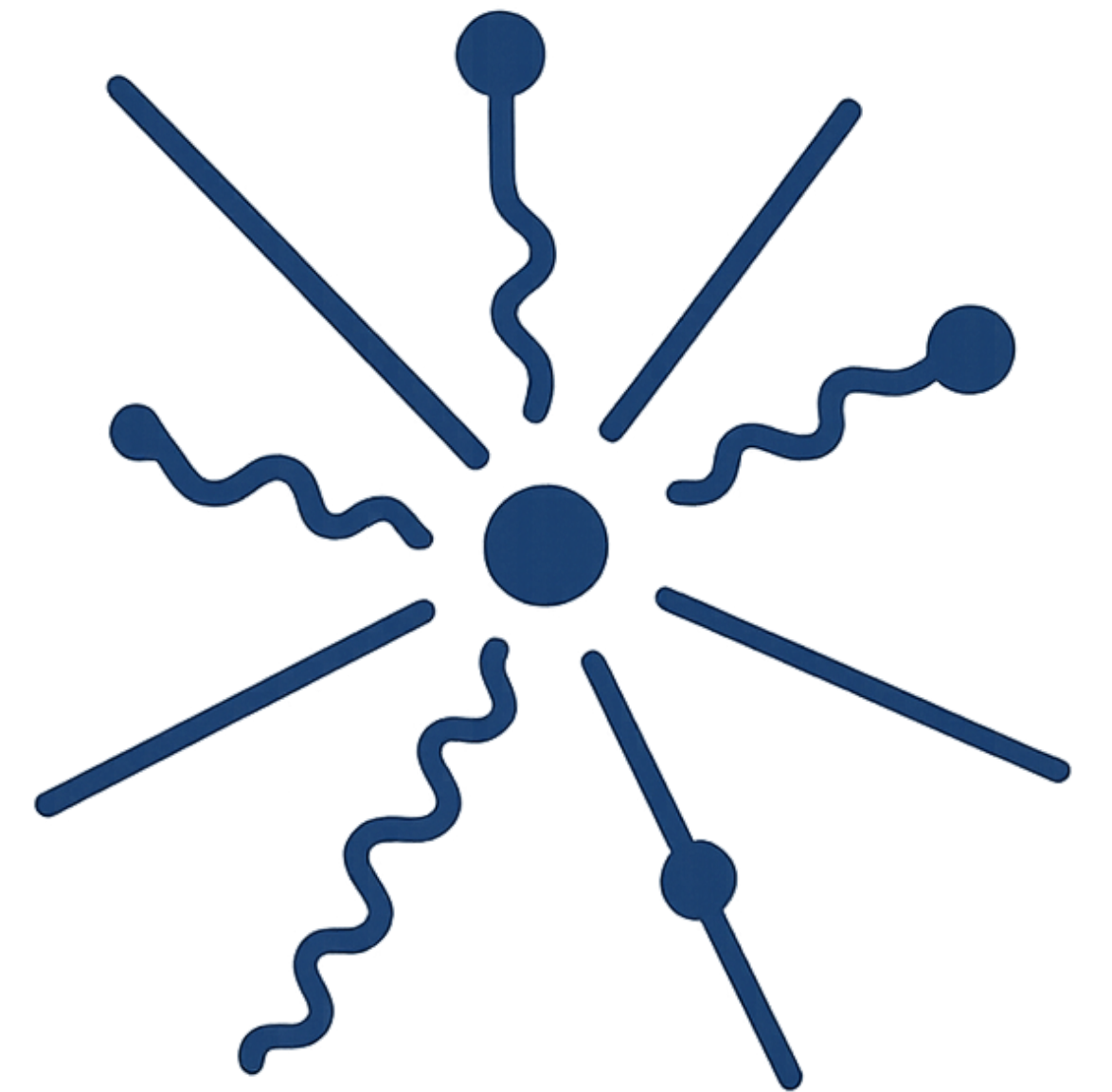
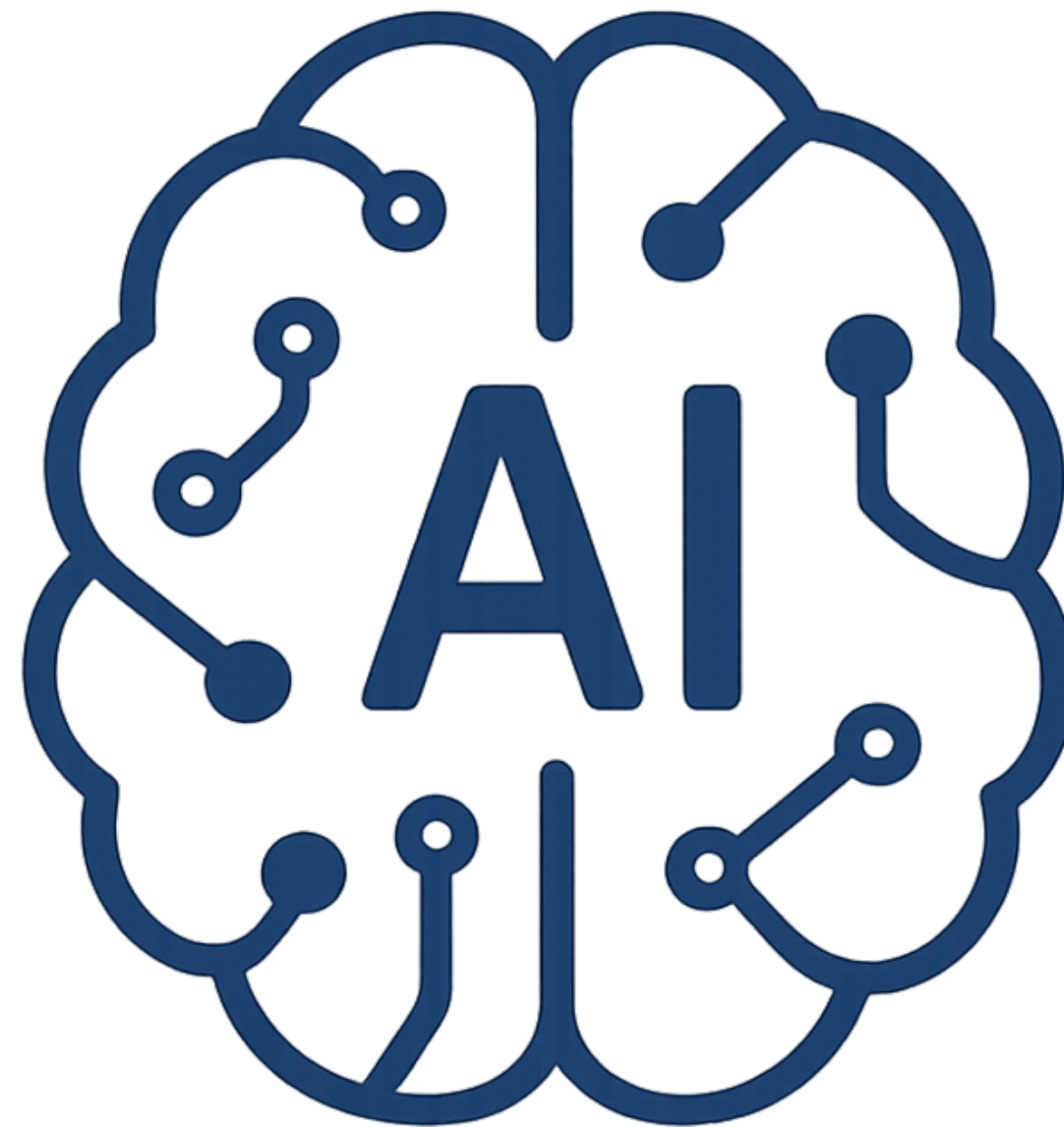
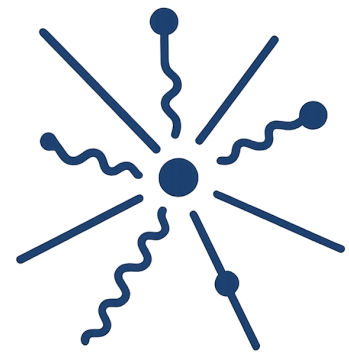
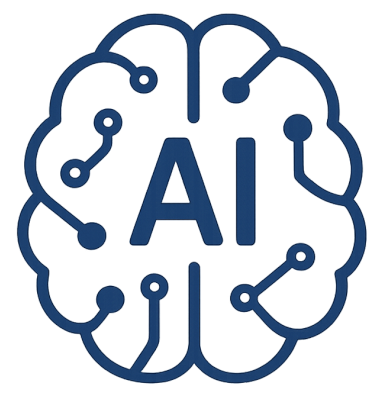


Introduction to

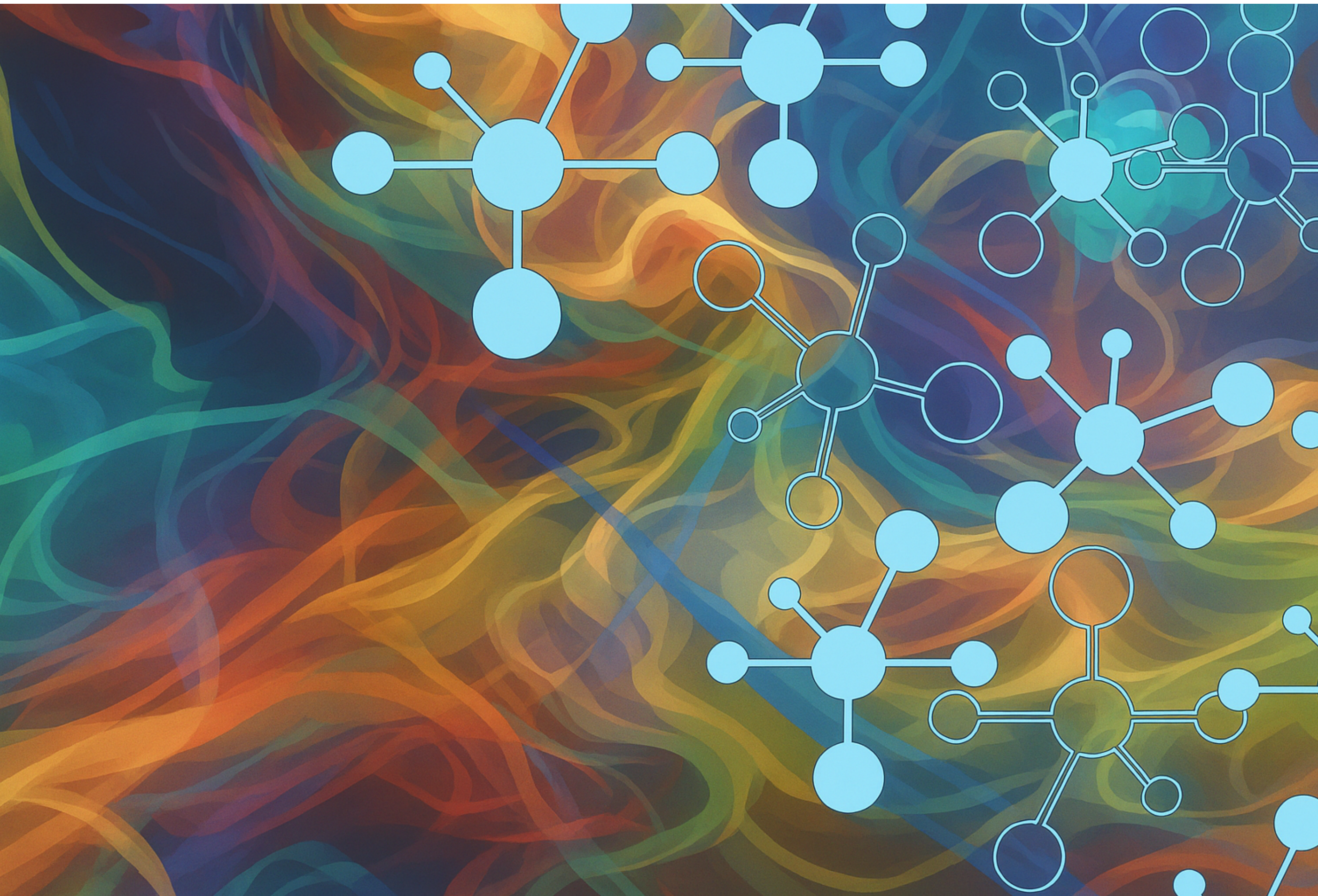
AI-Driven HEP

S. A. Fard - School of Physics (IPM)



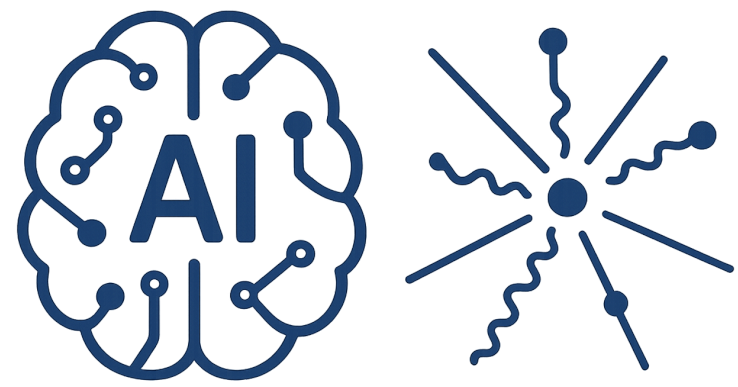


AI-Driven HEP 11



Session 11

Differentiable Programming

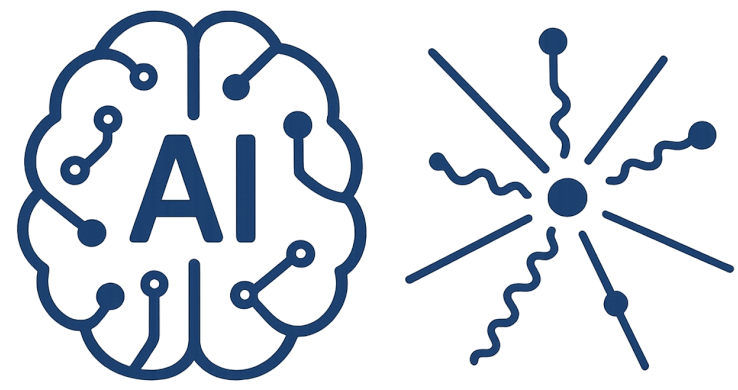


Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] <https://www.jmlr.org/papers/volume18/17-468/17-468.pdf>
- [3] Differentiable Matrix Elements with MadJax, Lukas Heinrich and Michael Kagan 2023 J. Phys.: Conf. Ser. **2438** 012137
(<https://iopscience.iop.org/article/10.1088/1742-6596/2438/1/012137>)
- [4] Machine Learning for Fundamental Physics School (ML4FP) 2025, <https://indico.physics.lbl.gov/event/3174/overview>
Special thanks to
Sean Gasiorowski (SLAC) and Sascha Diefenbacher (Berkeley)
- [5] <https://github.com/lukasheinrich/pyhep2020-autodiff-tutorial/blob/master/Tutorial.ipynb>
- [6] <https://theoryandpractice.org/stats-ds-book/intro.html>



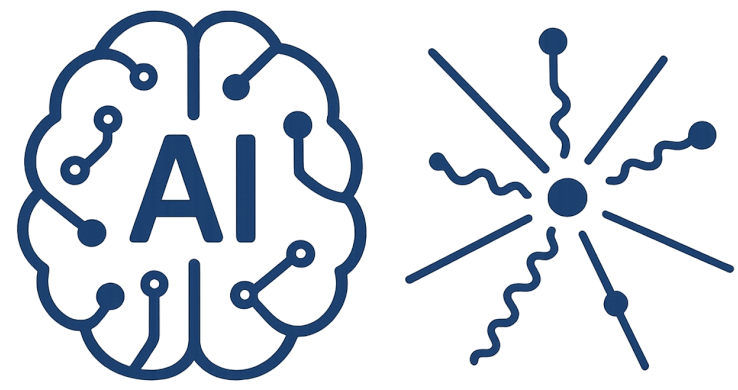
Derivatives in Computer Programs



AI-Driven HEP 11

Derivatives in Computer Programs

```
def f(a,b):  
    return math.log10(a * b)
```

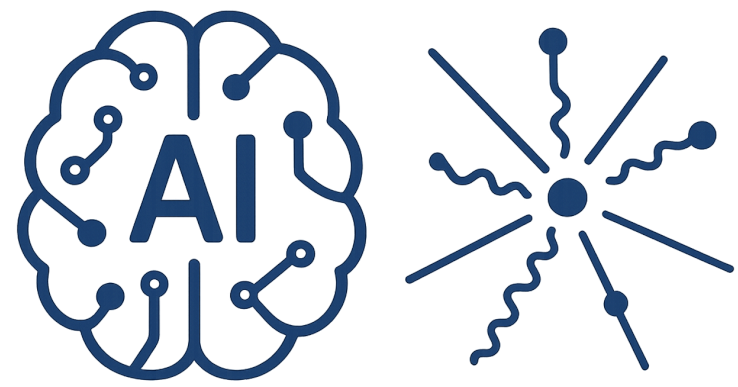
Derivatives in Computer Programs

1) Manually working out derivatives and coding them

2) Symbolic differentiation

3) Numerical differentiation using finite difference approximations

```
def f(a,b):  
    return math.log10(a * b)
```



Derivatives in Computer Programs

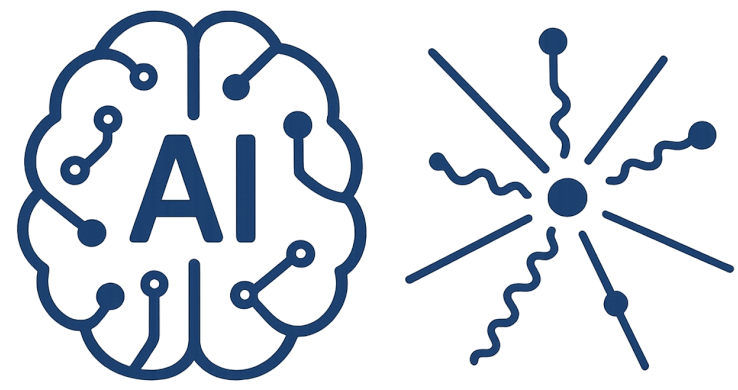
1) Manually working out derivatives and coding them

2) Symbolic differentiation

3) Numerical differentiation using finite difference approximations

4) Automatic Differentiation

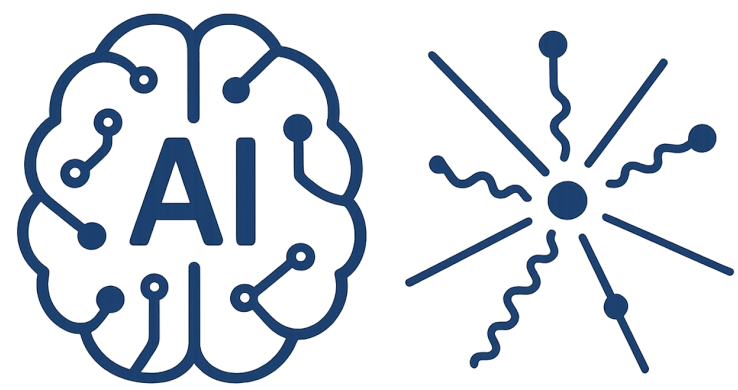
```
def f(a,b):  
    return math.log10(a * b)
```

Derivatives in Computer Programs

1) Manually working out derivatives
and coding them

```
def f(a,b):  
    return math.log10(a * b)  
  
    def dfda(a):  
        return 1 / a
```



Derivatives in Computer Programs

1) Manually working out derivatives
and coding them

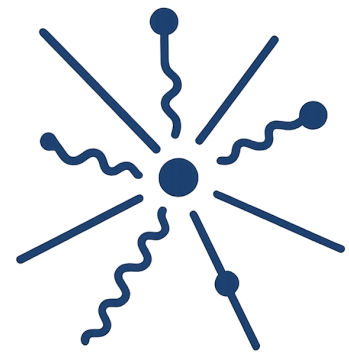
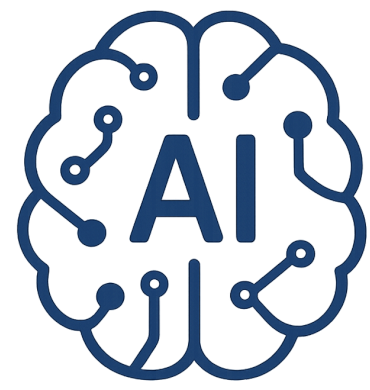
Time-Consuming – Deriving and coding gradients for
large or nested functions

Hard to Maintain – Any change in the original function
requires re-deriving and re-coding all derivatives.

Scalability – high-dimensional problems

Limited to analytic form – Manual derivatives are
tailored to a specific function, reducing code flexibility
and modularity.

```
def f(a,b):  
    return math.log10(a * b)  
  
def dfda(a):  
    return 1 / a
```

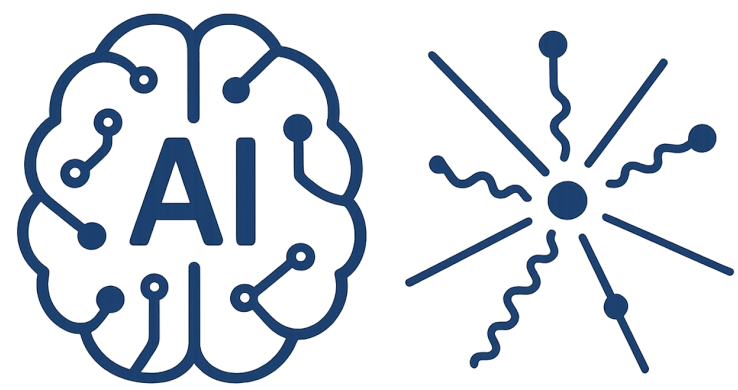
Derivatives in Computer Programs

2) Symbolic differentiation

e.g. Mathematica, SymPy

Gets messy/costly with number of terms

Only applicable to closed form expressions (no control flow)

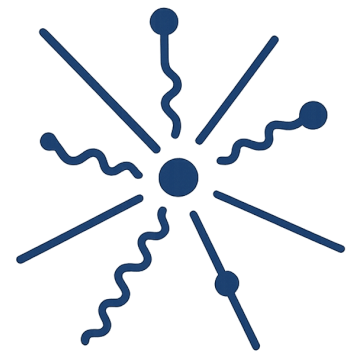
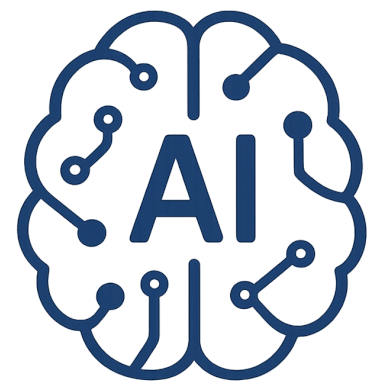


Derivatives in Computer Programs

2) Symbolic differentiation

Logistic map $l_{n+1} = 4l_n(1 - l_n)$, $l_1 = x$

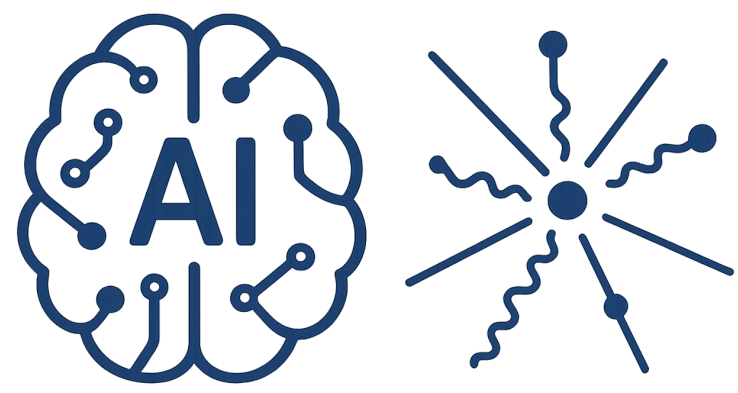
n	l_n	$\frac{d}{dx}l_n$	$\frac{d}{dx}l_n$ (Simplified form)
1	x	1	1
2	$4x(1 - x)$	$4(1 - x) - 4x$	$4 - 8x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2 - 16x(1-2x)^2 - 64x(1-x)(1-2x)$	$16(1 - 10x + 24x^2 - 16x^3)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$	$64(1 - 42x + 504x^2 - 2640x^3 + 7040x^4 - 9984x^5 + 7168x^6 - 2048x^7)$



Derivatives in Computer Programs

3) Numerical differentiation using
finite difference approximations

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}, 0 < h \ll 1$$

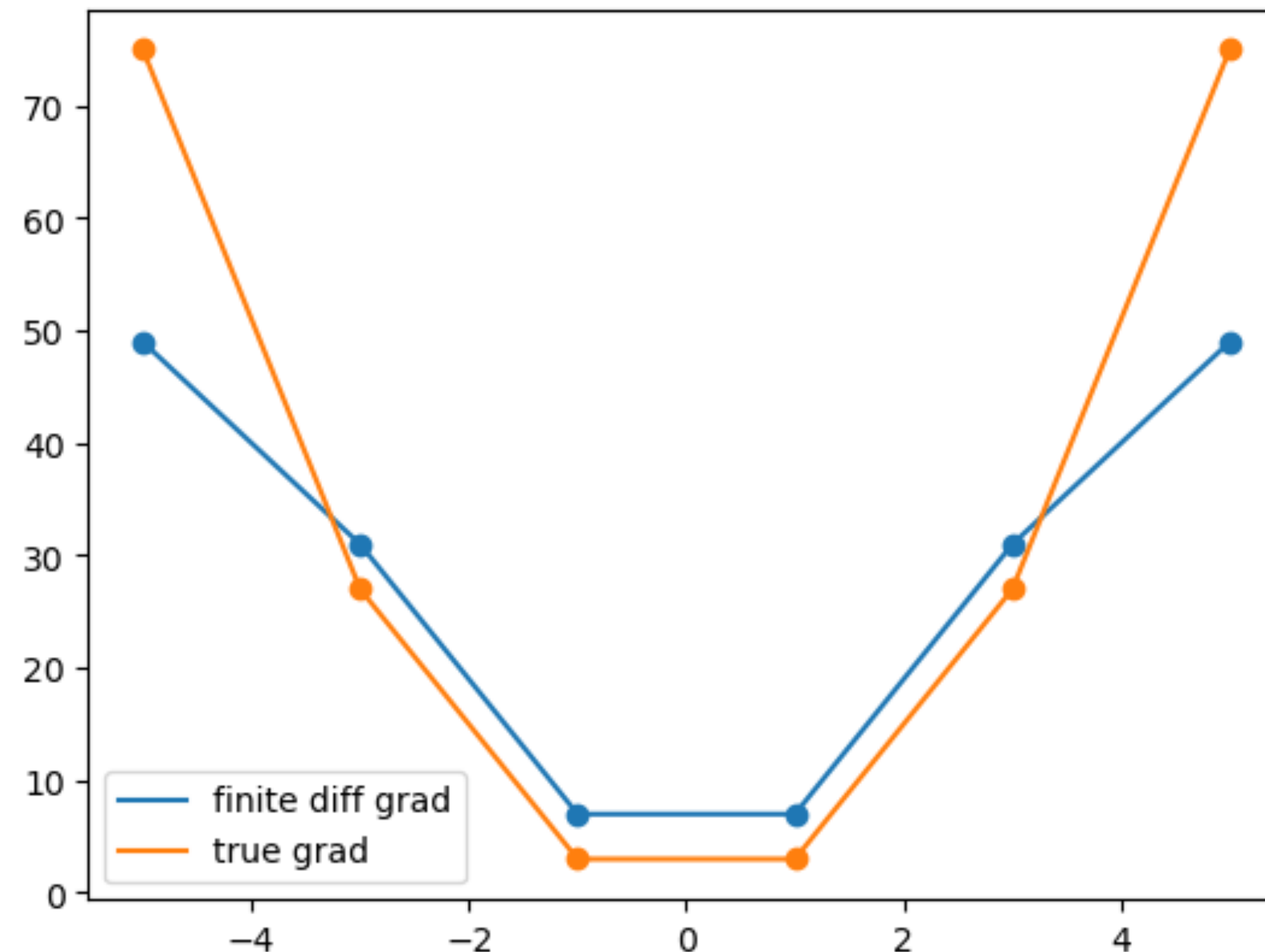


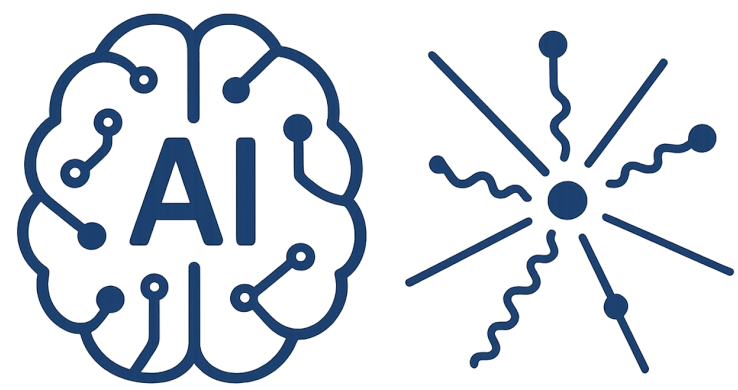
Derivatives in Computer Programs

3) Numerical differentiation using finite difference approximations

```
def black_box_func(x):  
    return x**3  
  
def true_gradient_func(x):  
    return 3*x**2
```

$$\Delta x \sim 2$$



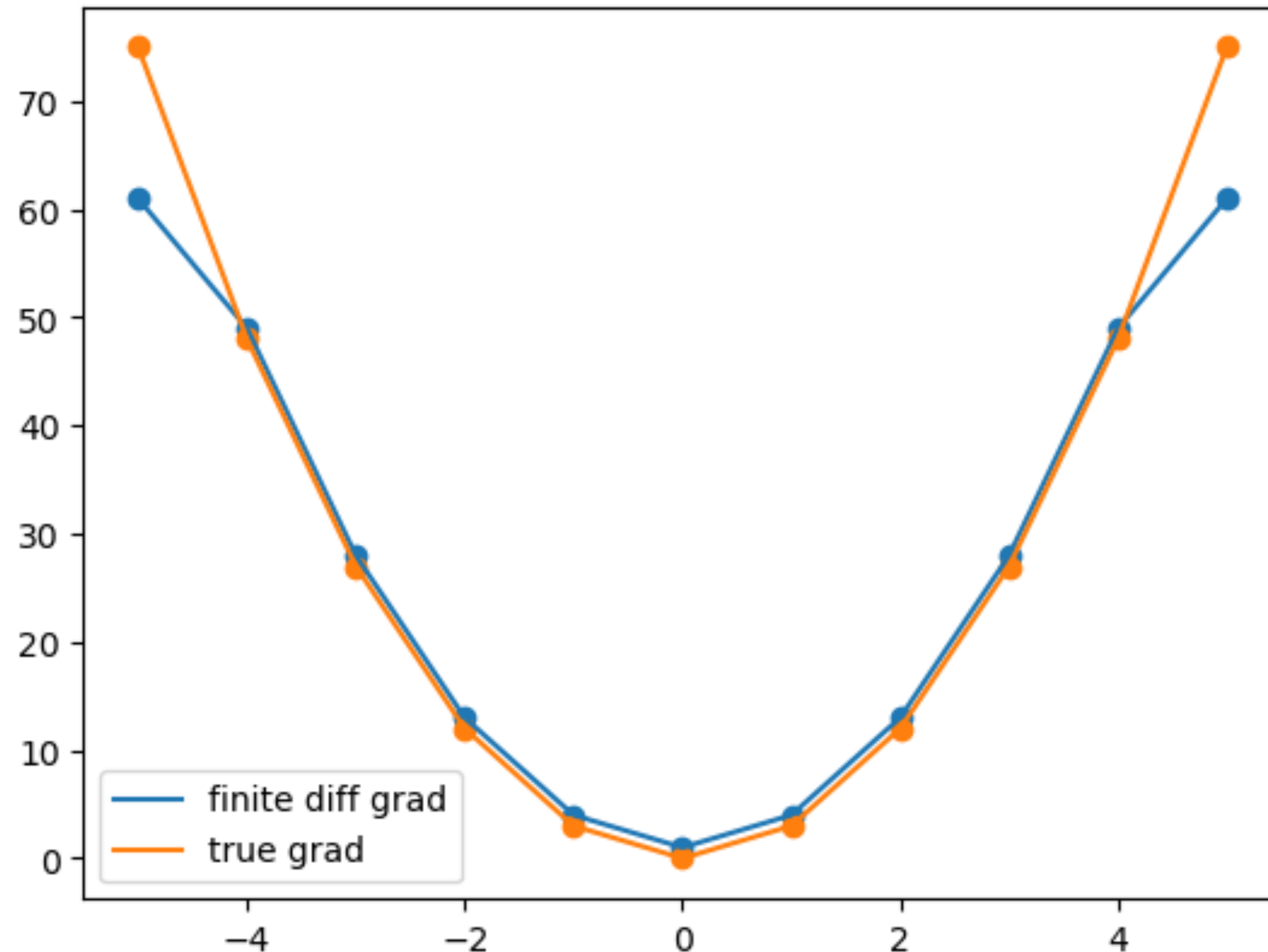


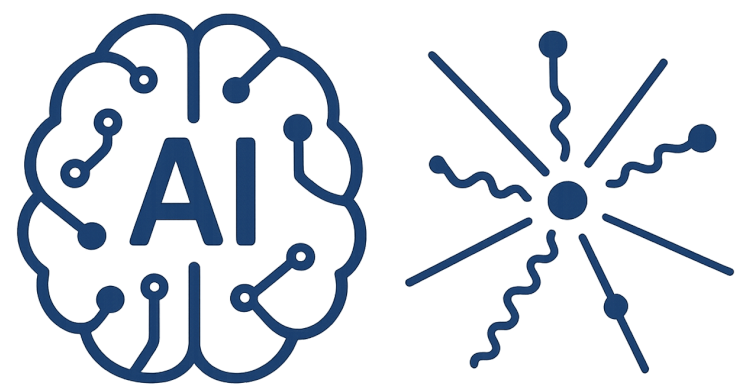
Derivatives in Computer Programs

3) Numerical differentiation using finite difference approximations

```
def black_box_func(x):  
    return x**3  
  
def true_gradient_func(x):  
    return 3*x**2
```

$$\Delta x \sim 1$$



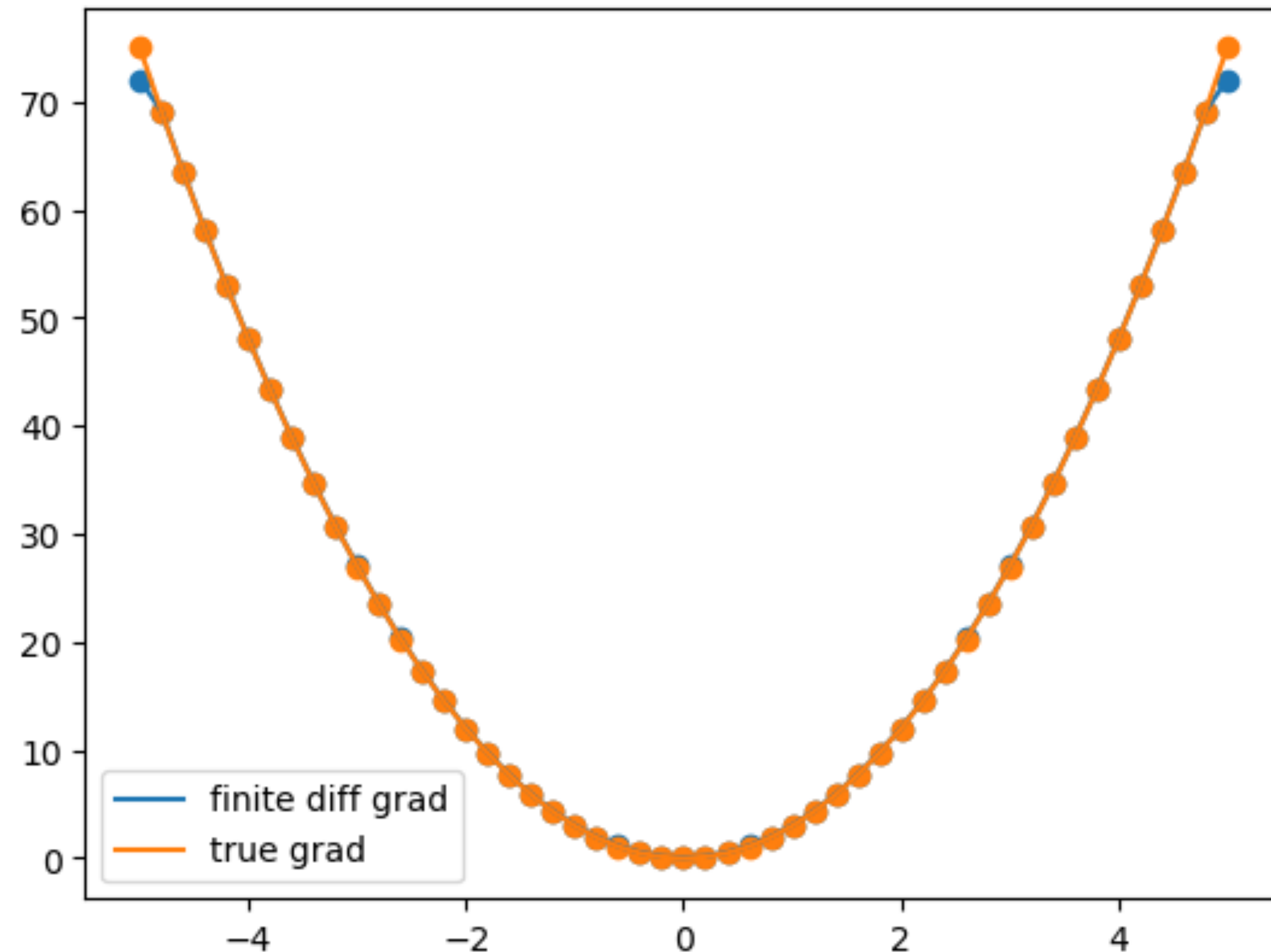


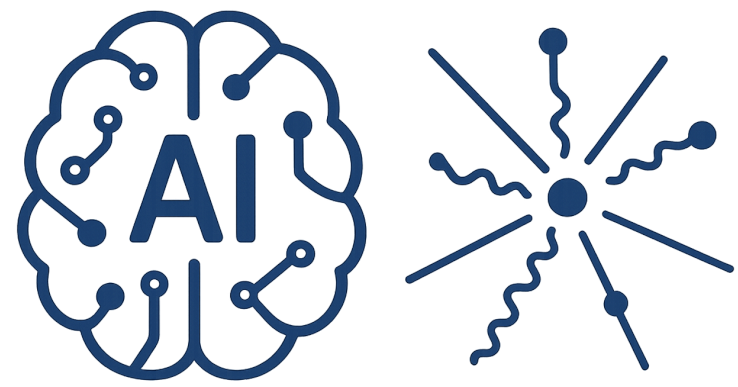
Derivatives in Computer Programs

3) Numerical differentiation using finite difference approximations

$$\Delta x \sim 0.1$$

```
def black_box_func(x):  
    return x**3  
  
def true_gradient_func(x):  
    return 3*x**2
```





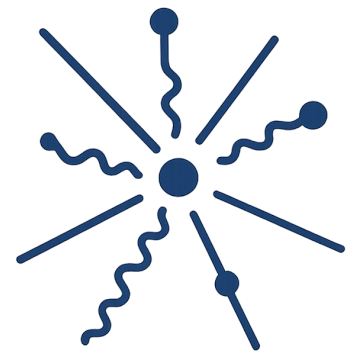
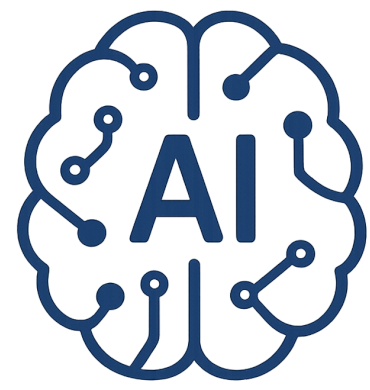
Derivatives in Computer Programs

3) Numerical differentiation using
finite difference approximations

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h}, 0 < h \ll 1$$

Blows up with input dimensionality (one function eval per basis vector)

Approximation errors from choices of h (Rounding Error & Truncation Error)

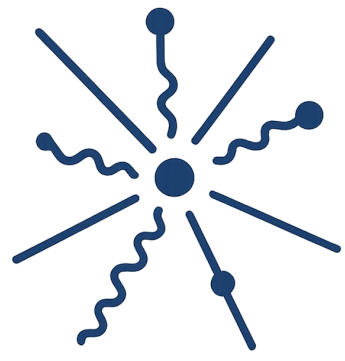
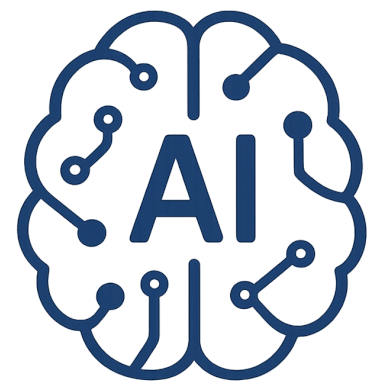


Automatic Differentiation

Break down arbitrary computer program into a graph of fundamental operations with known derivatives

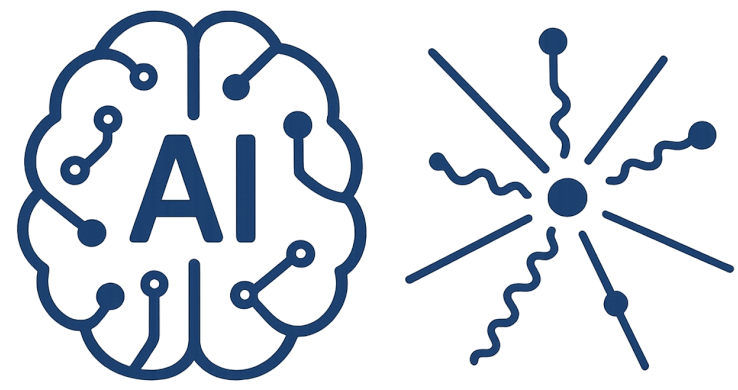
Exact gradient calculation, broadly applicable

Scales well! Gradient cost $\sim$ original code cost



Automatic Differentiation

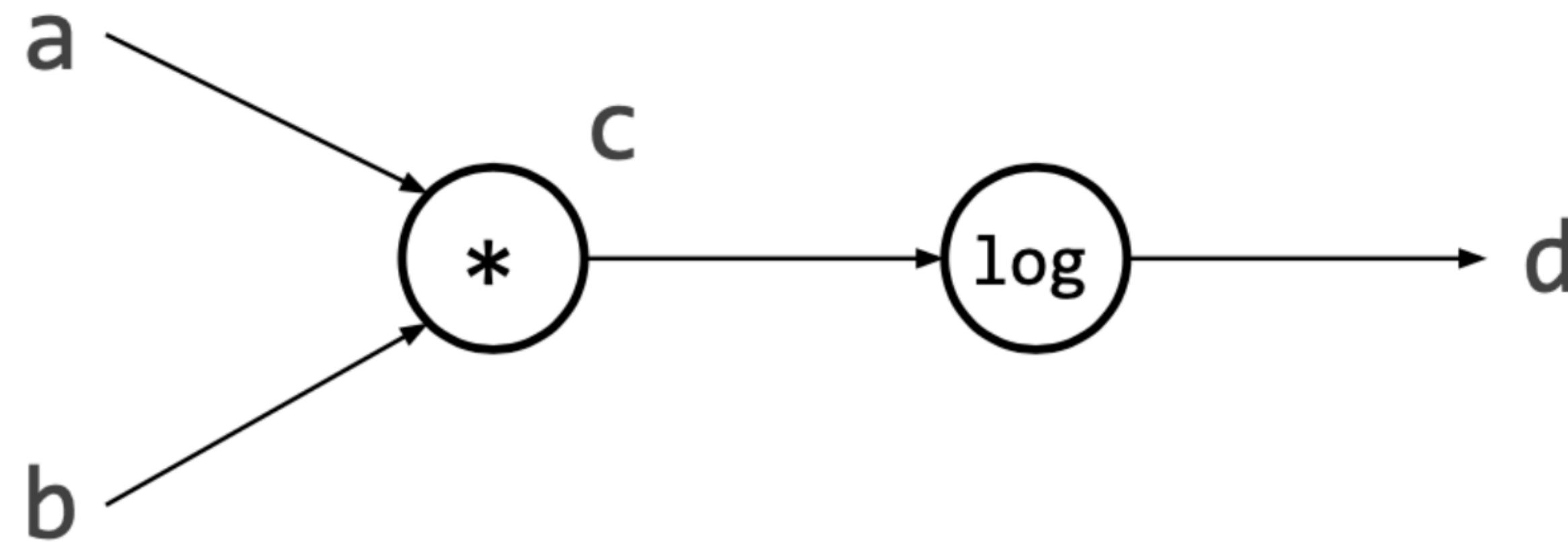
```
def f(a,b):  
    return math.log10(a * b)
```

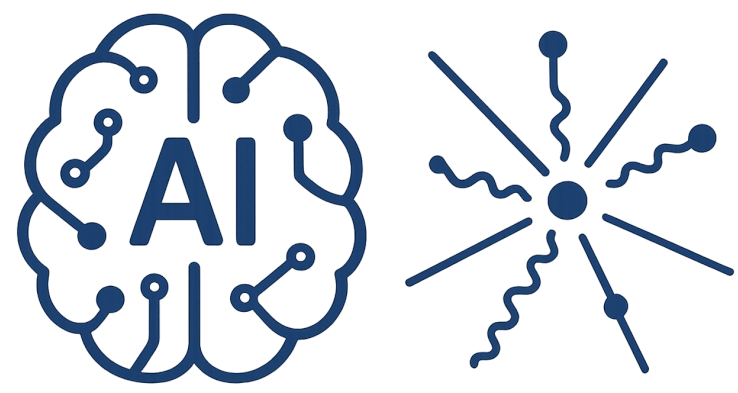



Automatic Differentiation

```
def f(a,b):  
    return math.log10(a * b)
```

Computational Graph

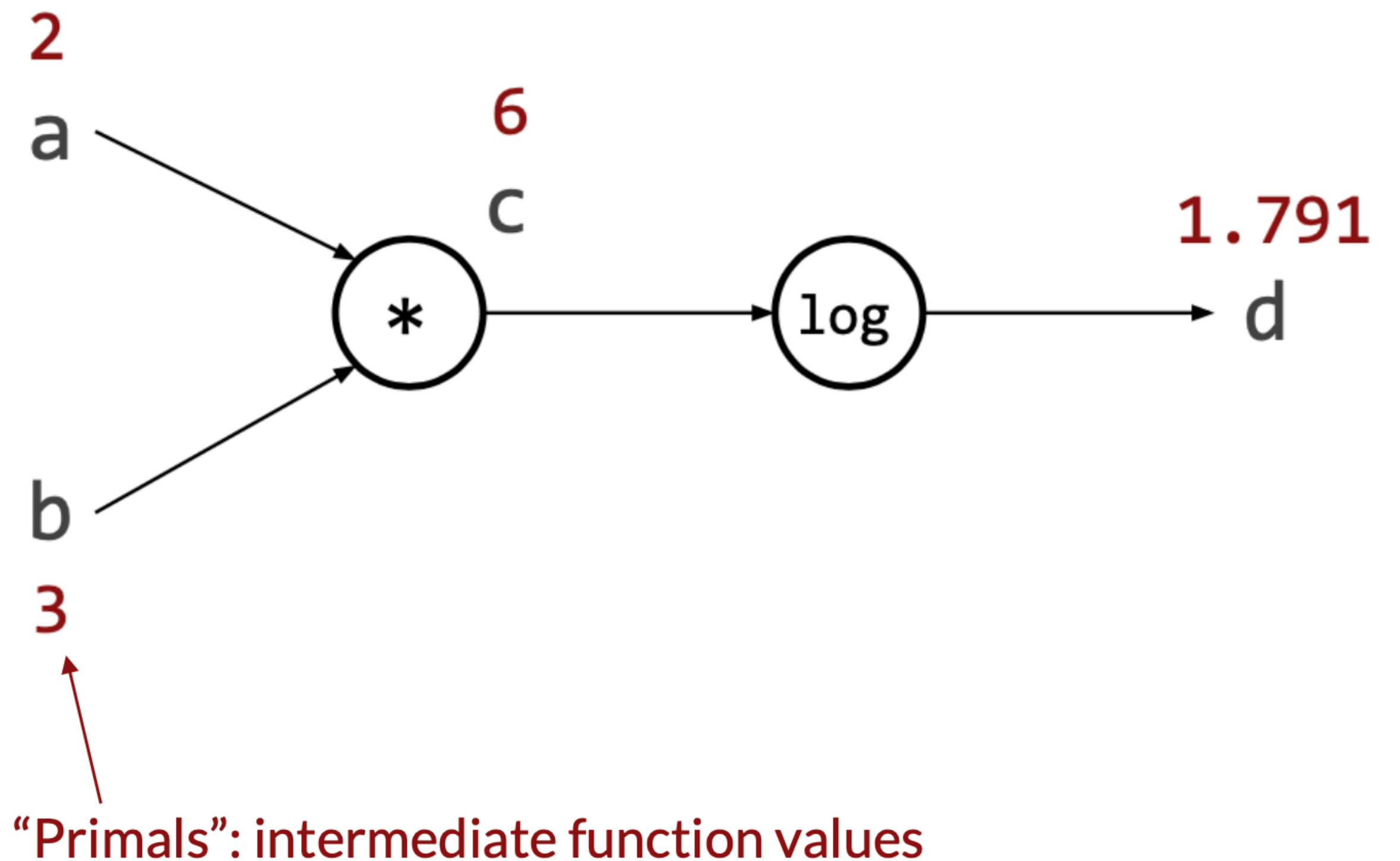


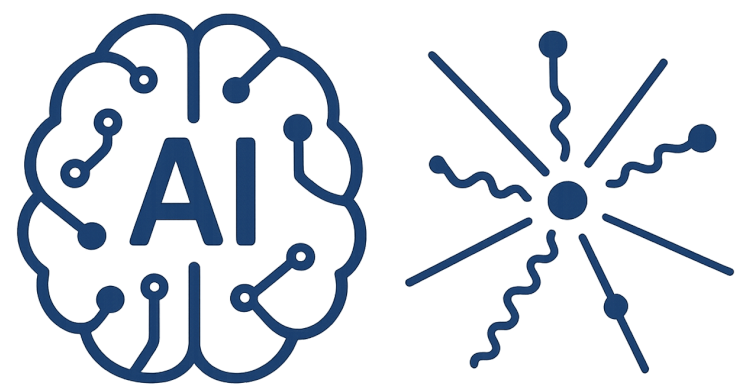


Forward Mode

```
def f(a,b):  
    return math.log10(a * b)
```

$f(2, 3) = 1.791$



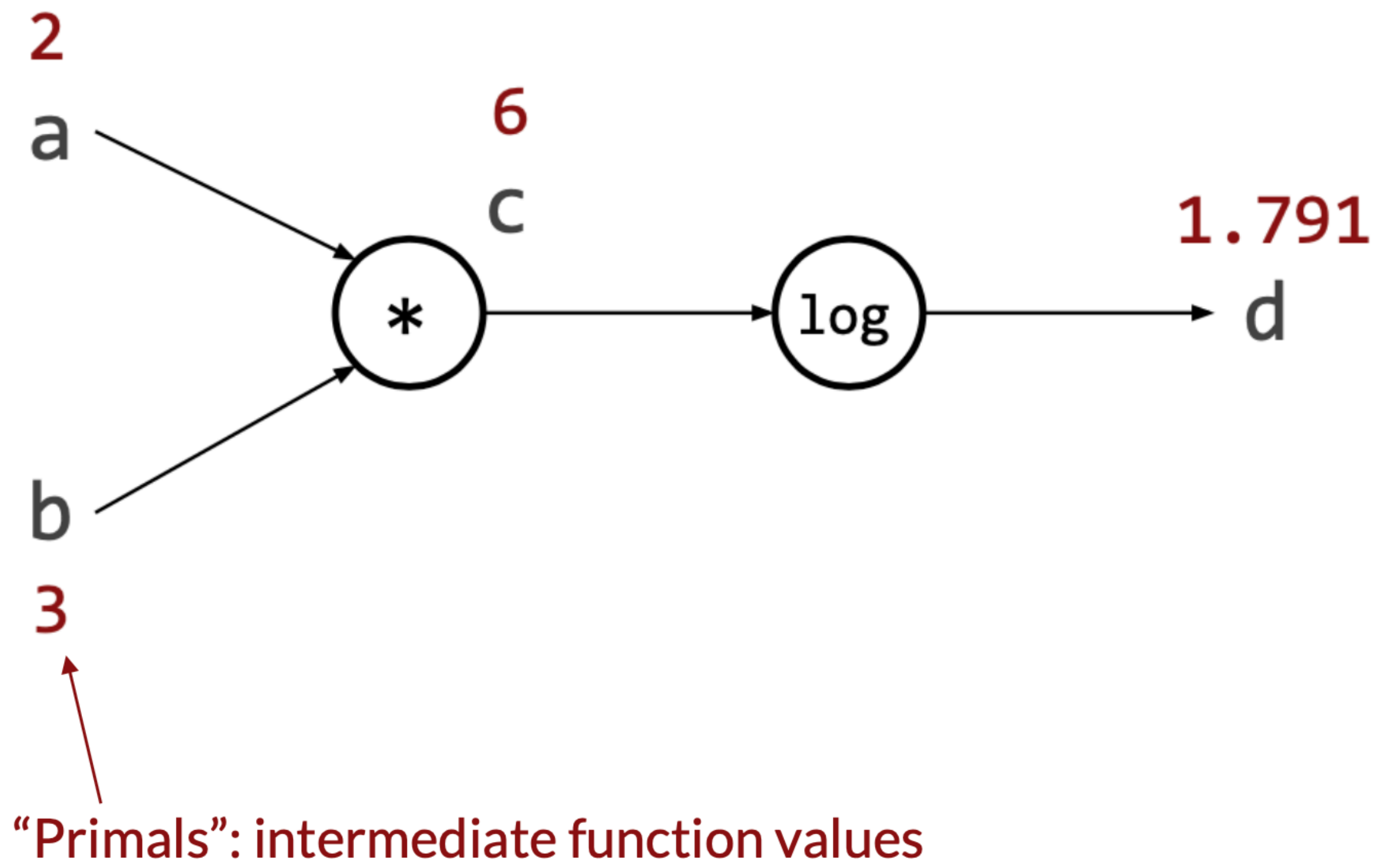


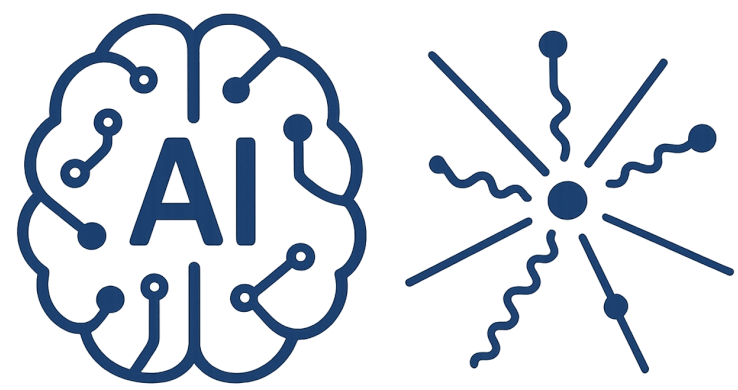
Forward Mode

```
def f(a,b):  
    return math.log10(a * b)
```

$$f(2, 3) = 1.791$$

Normal (forward) evaluation of the code for values of a, b results in a set of intermediate values (**primals**) at each stage of the computation



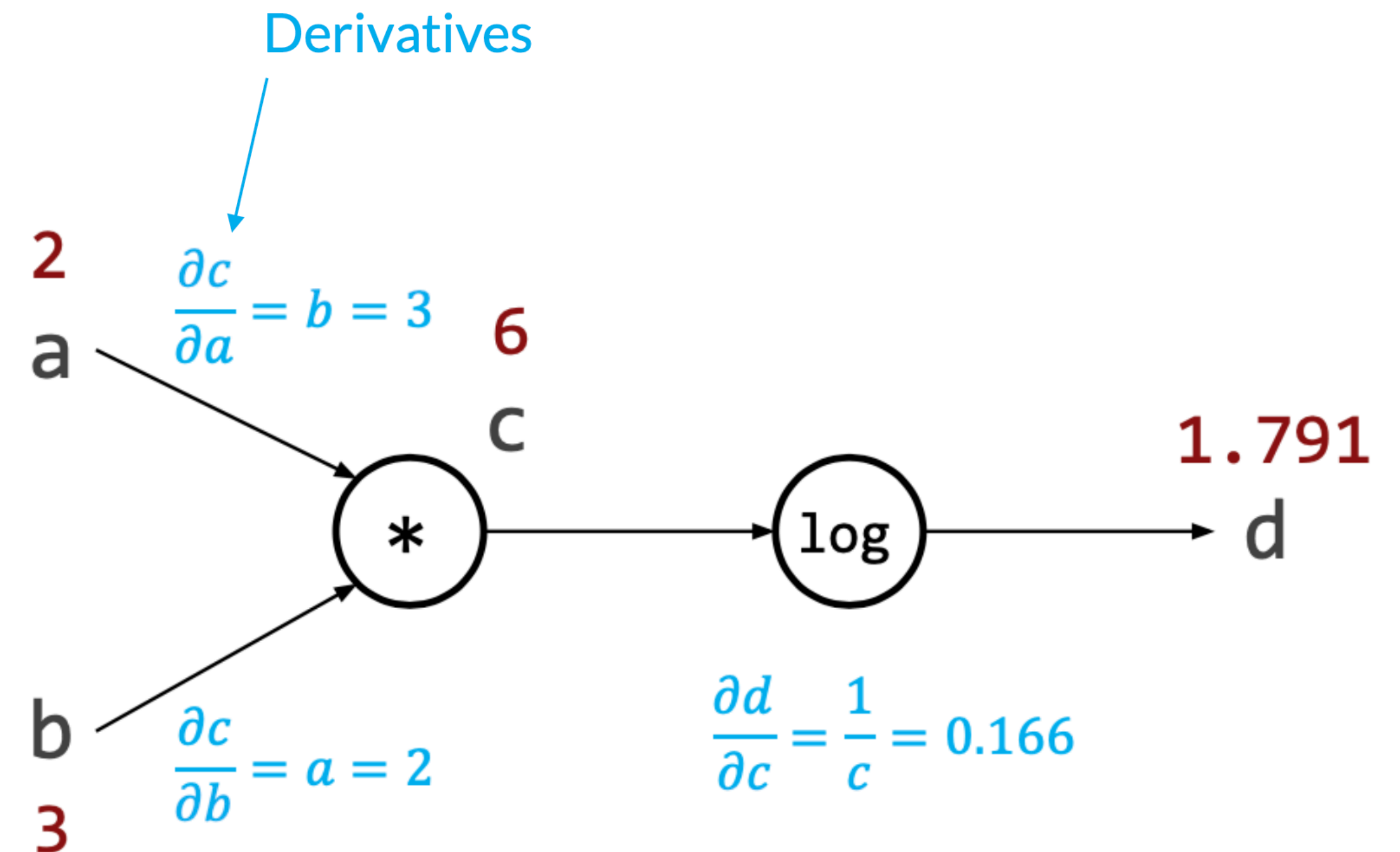


Forward Mode AD

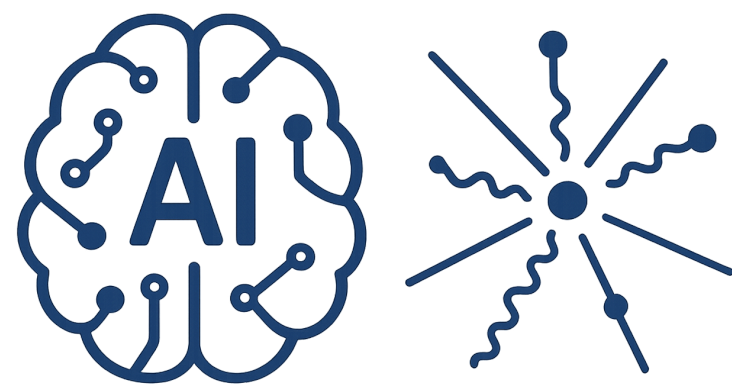
```
def f(a,b):  
    return math.log10(a * b)
```

df(2,3) = [0.5, 0.333]

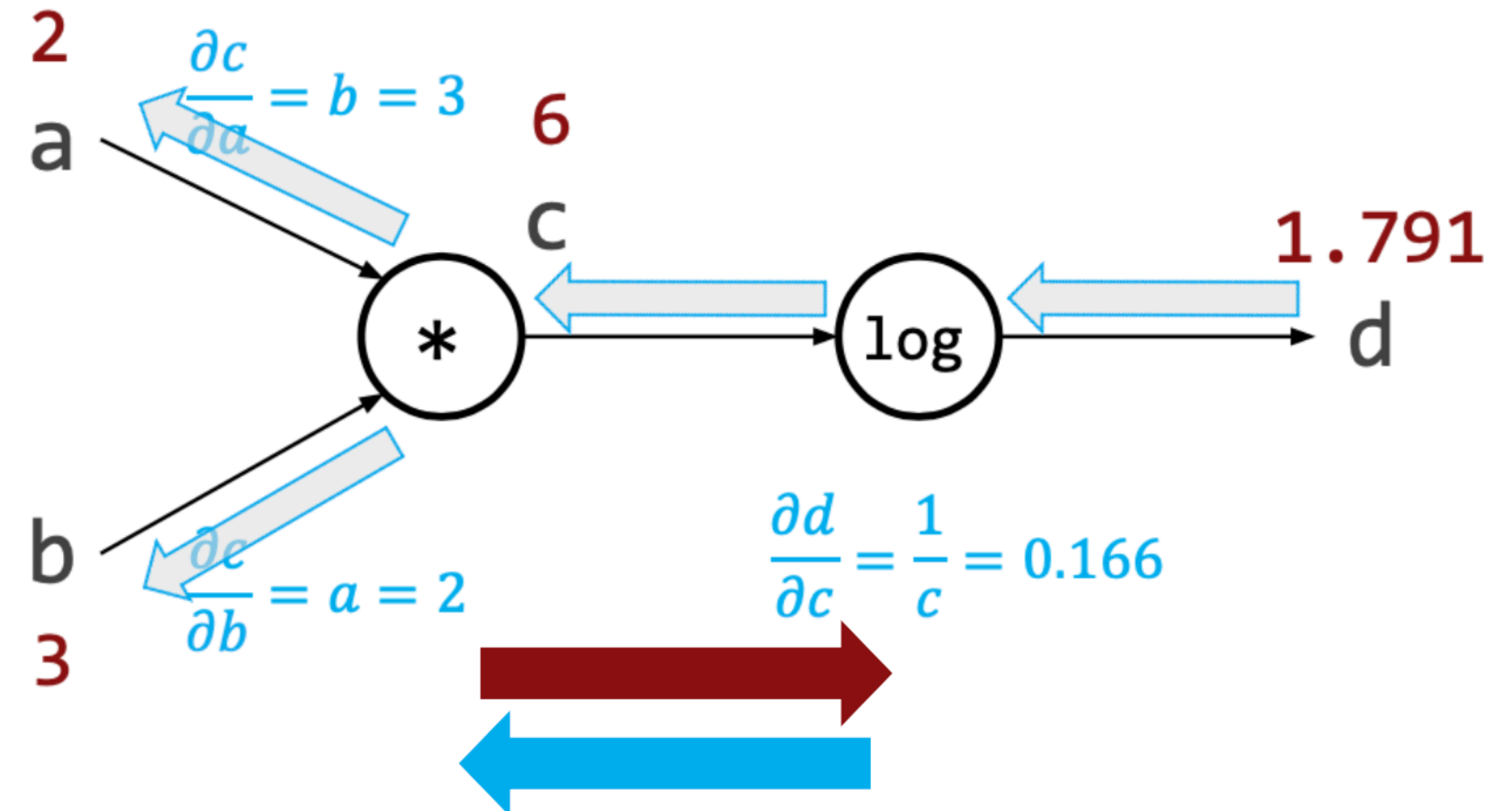
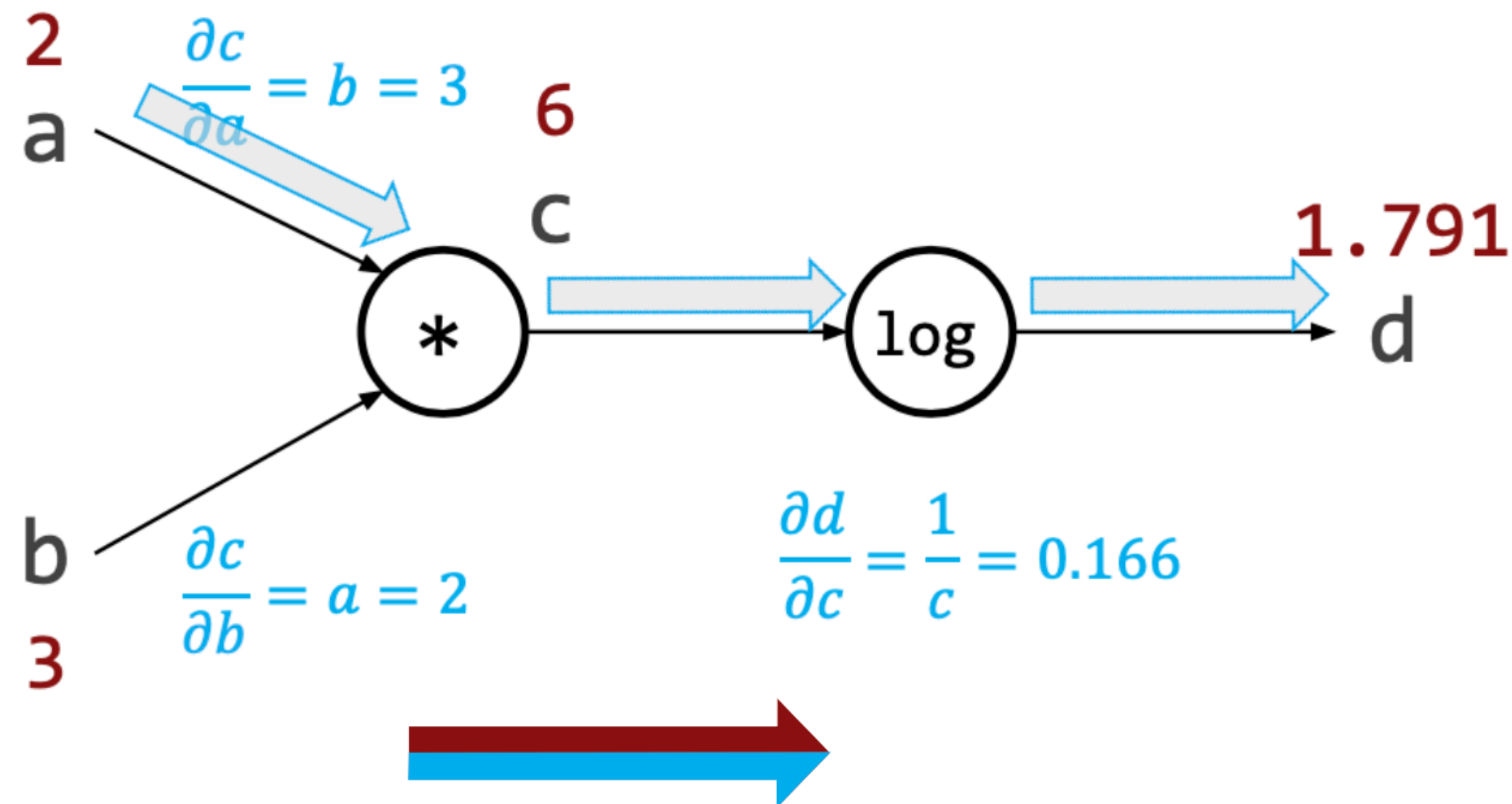
Normal (forward) evaluation of the code for values of a, b results in a set of intermediate values (**primals**) at each stage of the computation



Chain Rule: $\frac{\partial d}{\partial a} = \frac{\partial d}{\partial c} \frac{\partial c}{\partial a} = 0.166 * 3 = 0.5$



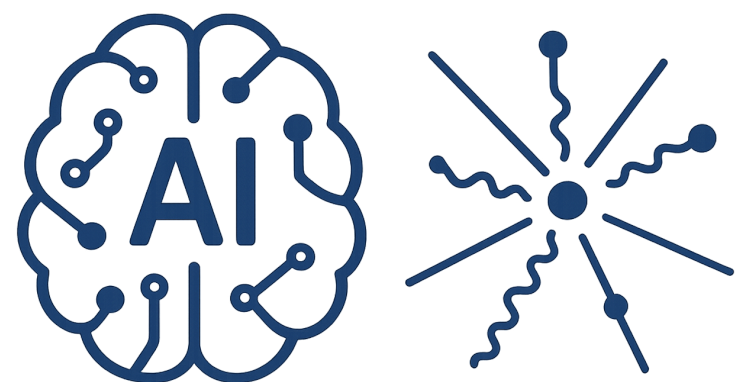
Forward & Reverse AD





Example

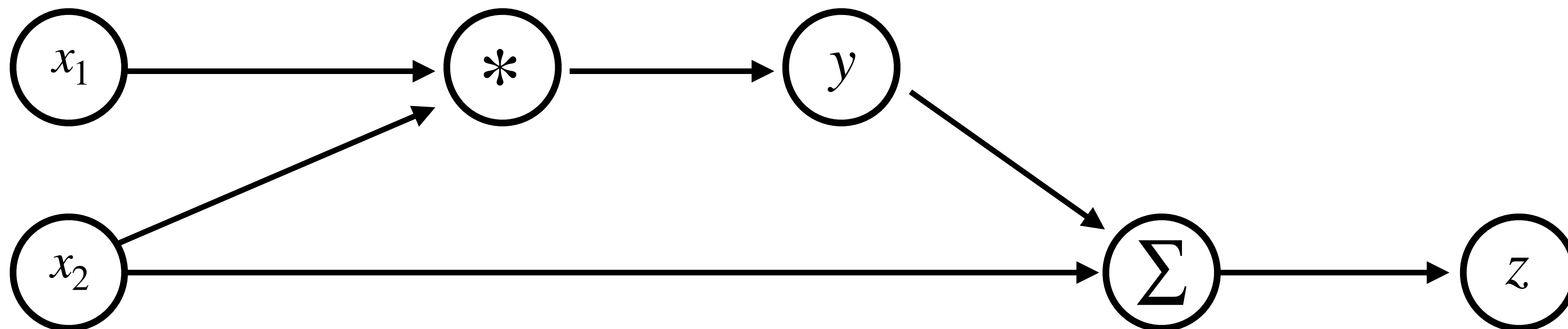
$$z = x_1 x_2 + x_2$$

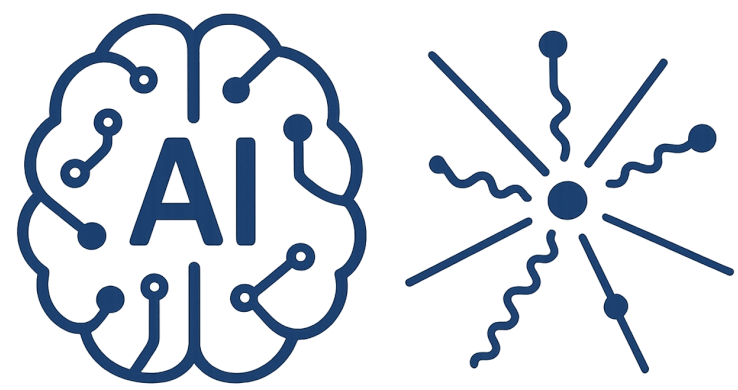


Example

$$z = x_1 x_2 + x_2$$

Computational Graph

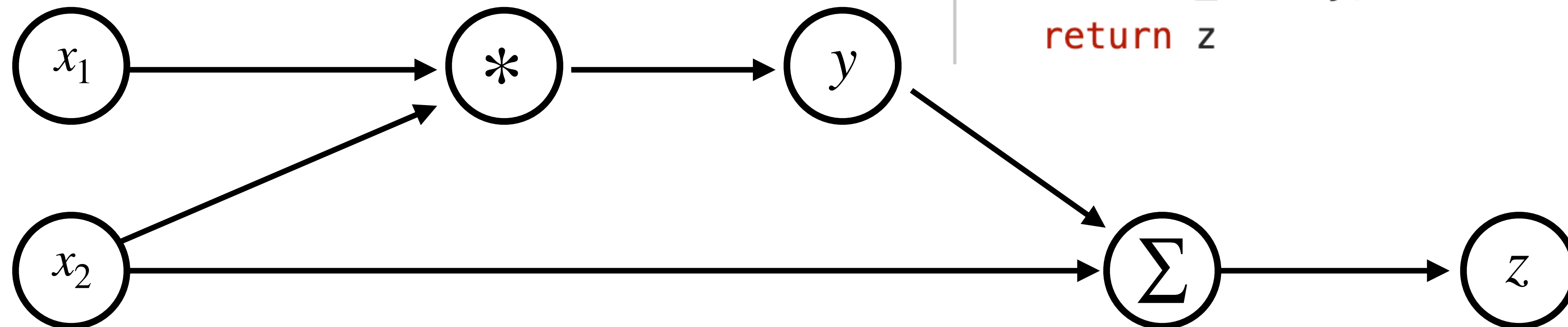




Example

Forward

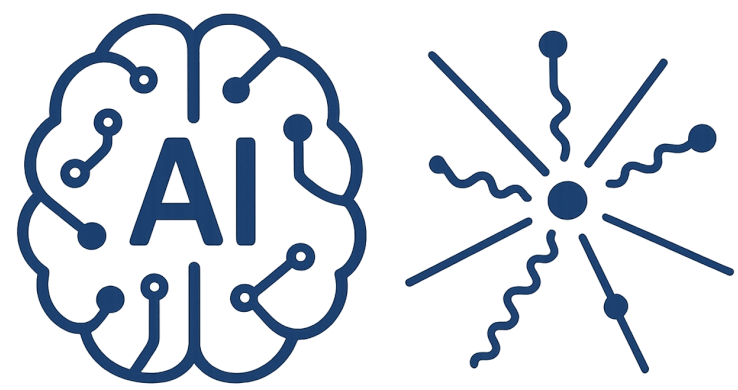
$$z = x_1 x_2 + x_2$$



```
✓ def mul_func(x1,x2):  
    | return x1*x2
```

```
✓ def sum_func(x1,x2):  
    | return x1+x2
```

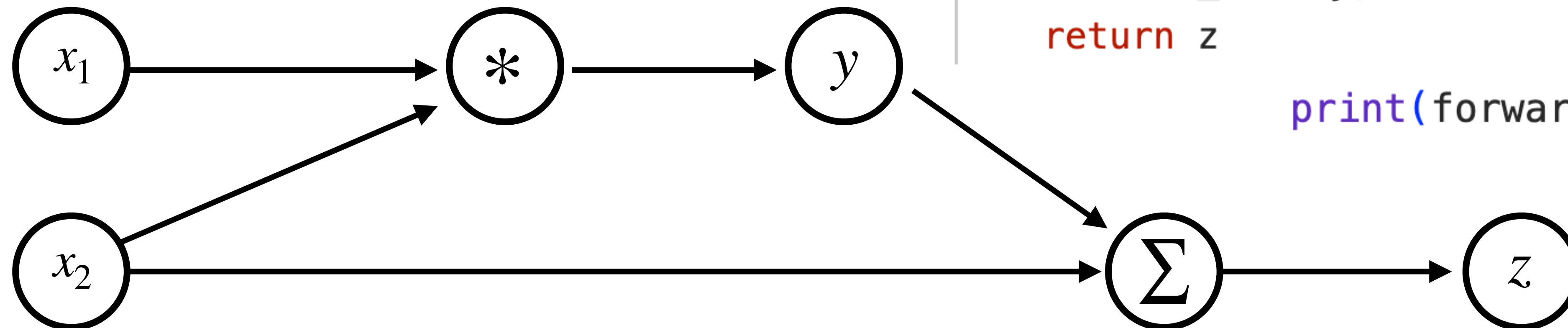
```
✓ def forward(x):  
    | x1,x2 = x  
    | y = mul_func(x1,x2)  
    | z = sum_func(y,x2)  
    | return z
```

Example

Forward

$$z = x_1 x_2 + x_2$$

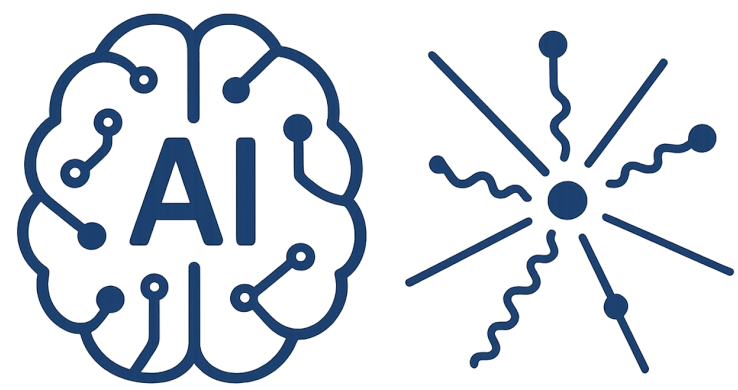


```
✓ def mul_func(x1,x2):  
    | return x1*x2
```

```
✓ def sum_func(x1,x2):  
    | return x1+x2
```

```
✓ def forward(x):  
    x1,x2 = x  
    y = mul_func(x1,x2)  
    z = sum_func(y,x2)  
    return z
```

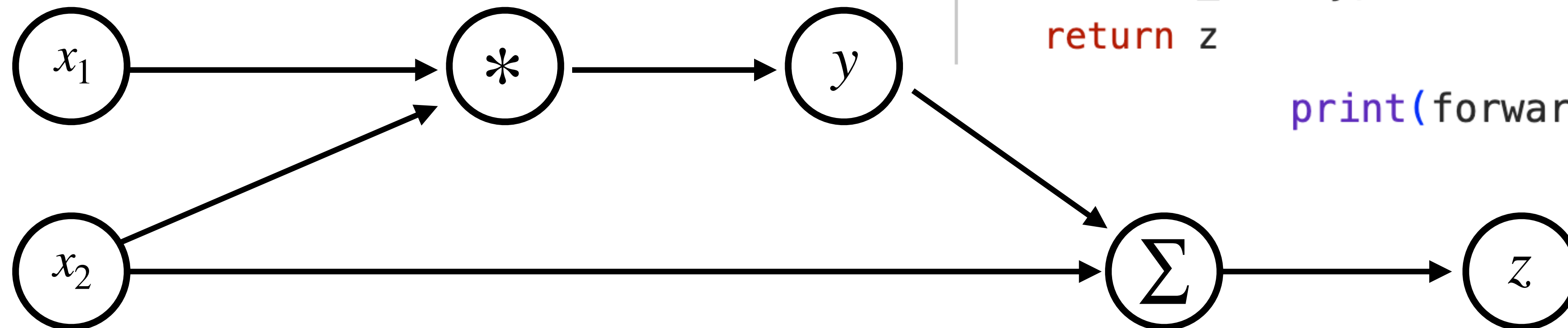
```
print(forward([2,4])) ?
```



Example

Forward

$$z = x_1 x_2 + x_2$$

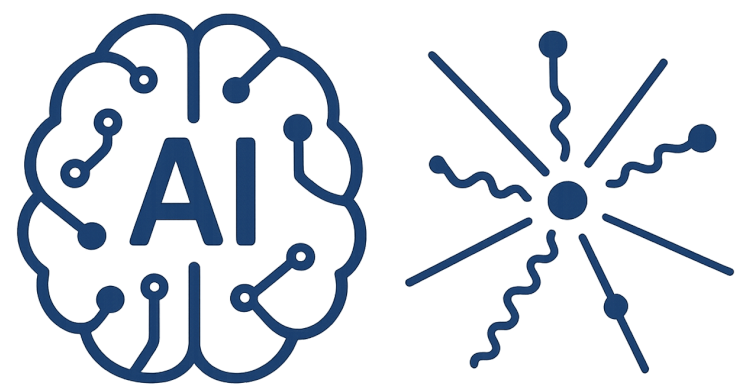


```
✓ def mul_func(x1,x2):  
    | return x1*x2
```

```
✓ def sum_func(x1,x2):  
    | return x1+x2
```

```
✓ def forward(x):  
    x1,x2 = x  
    y = mul_func(x1,x2)  
    z = sum_func(y,x2)  
    return z
```

```
print(forward([2,4])) 12
```

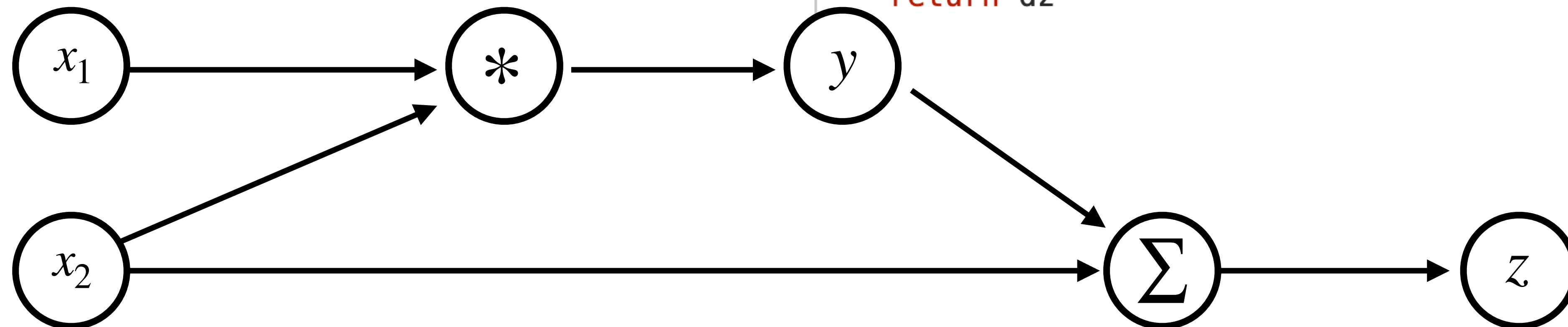


AI-Driven HEP 11

Example

Forward Mode

$$z = x_1 x_2 + x_2$$



```
def mul_jvp(x1,dx1,x2,dx2):
```

```
    dy = x1*dx2 + x2*dx1
```

```
    return dy
```

```
def sum_jvp(dx1,dx2):
```

```
    return dx1 + dx2
```

```
def forward_jvp(x,dx):
```

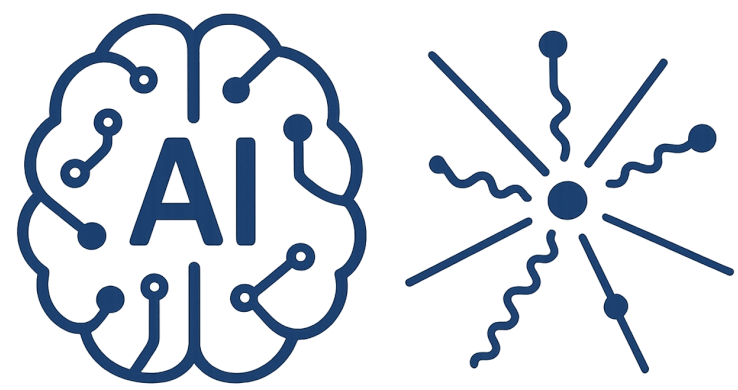
```
    x1,x2 = x
```

```
    dx1,dx2 = dx
```

```
    dy = mul_jvp(x1,dx1,x2,dx2)
```

```
    dz = sum_jvp(dy,dx2)
```

```
    return dz
```

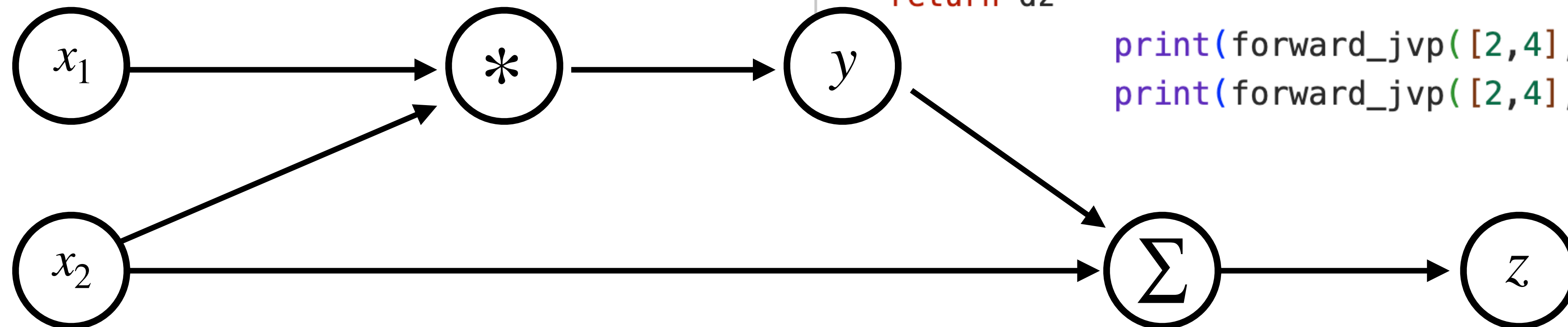



AI-Driven HEP 11

Example

Forward Mode

$$z = x_1 x_2 + x_2$$



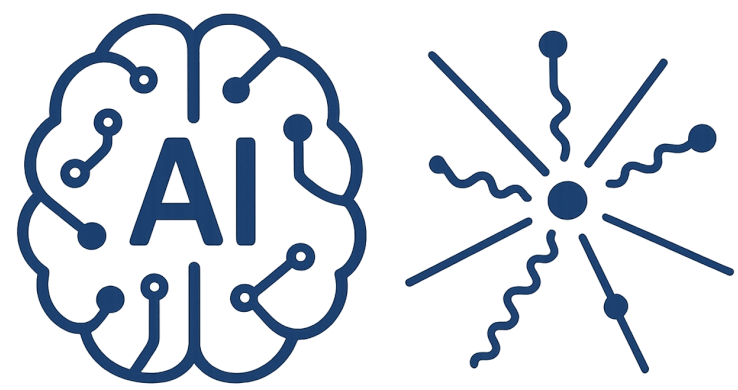
```
def mul_jvp(x1,dx1,x2,dx2):  
    dy = x1*dx2 + x2*dx1  
    return dy
```

```
def sum_jvp(dx1,dx2):  
    return dx1 + dx2
```

```
def forward_jvp(x,dx):  
    x1,x2 = x  
    dx1,dx2 = dx  
    dy = mul_jvp(x1,dx1,x2,dx2)  
    dz = sum_jvp(dy,dx2)  
    return dz
```

```
print(forward_jvp([2,4],[1,0]))  
print(forward_jvp([2,4],[0,1]))
```

?

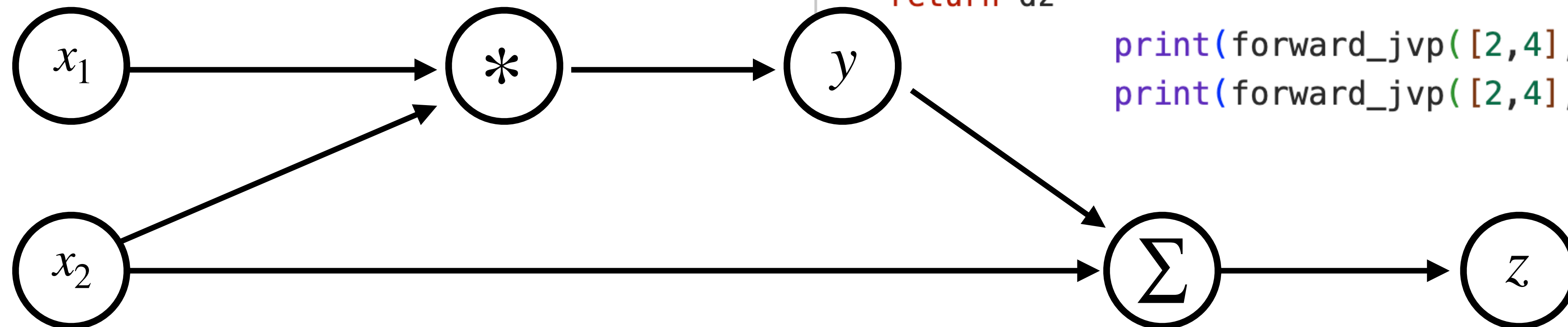


AI-Driven HEP 11

Example

Forward Mode

$$z = x_1 x_2 + x_2$$

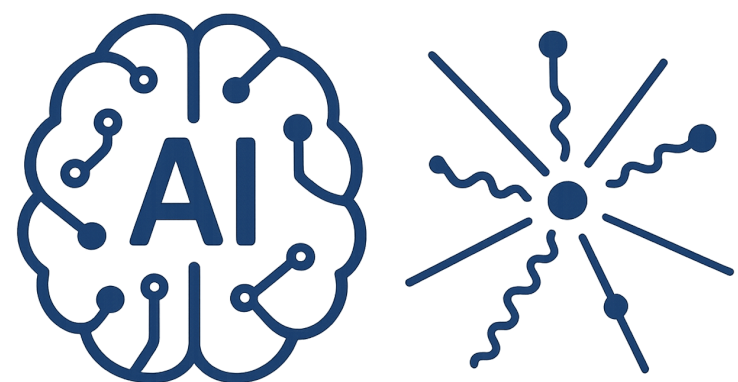


```
def mul_jvp(x1,dx1,x2,dx2):  
    dy = x1*dx2 + x2*dx1  
    return dy
```

```
def sum_jvp(dx1,dx2):  
    return dx1 + dx2
```

```
def forward_jvp(x,dx):  
    x1,x2 = x  
    dx1,dx2 = dx  
    dy = mul_jvp(x1,dx1,x2,dx2)  
    dz = sum_jvp(dy,dx2)  
    return dz
```

```
print(forward_jvp([2,4],[1,0])) 4  
print(forward_jvp([2,4],[0,1])) 3
```

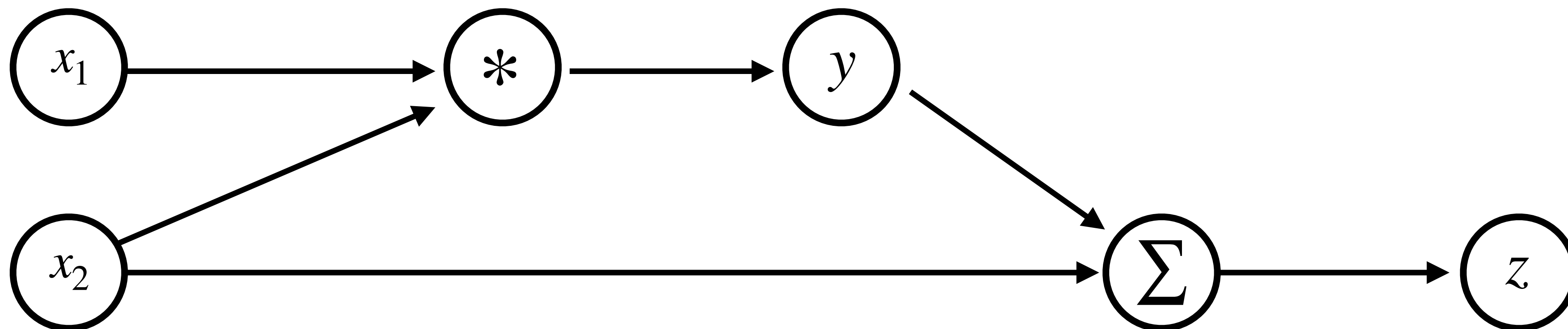


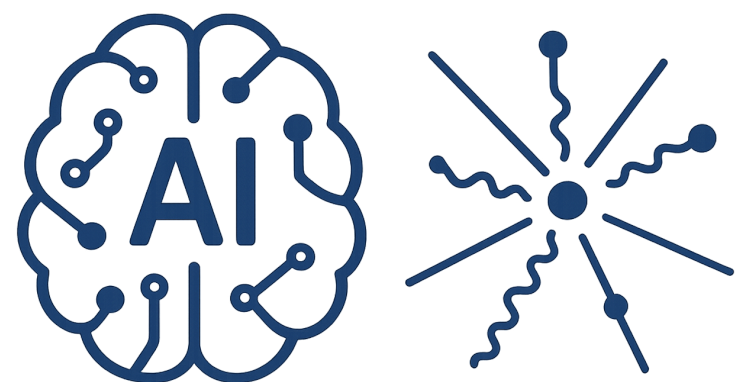
Example

Backward Mode

$$z = y + x_2$$

$$z = x_1 x_2 + x_2$$





Example

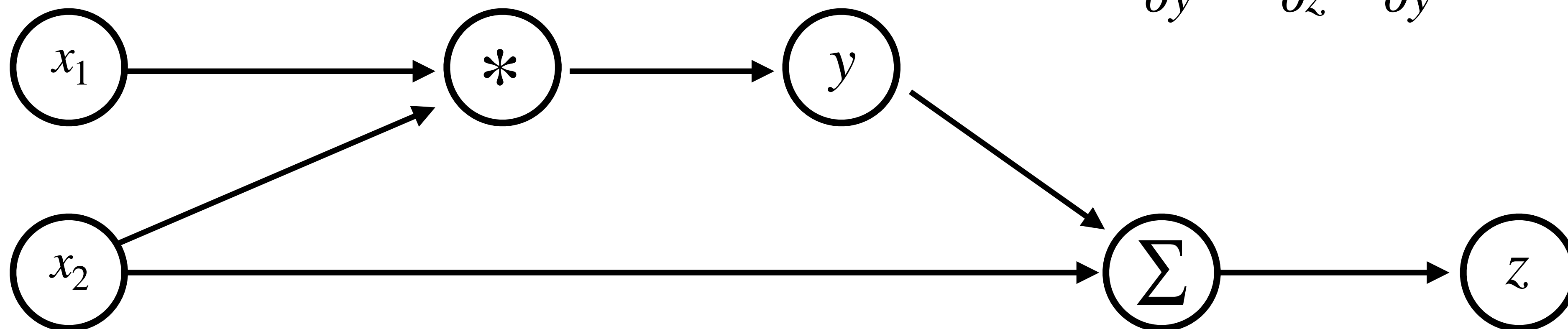
Backward Mode

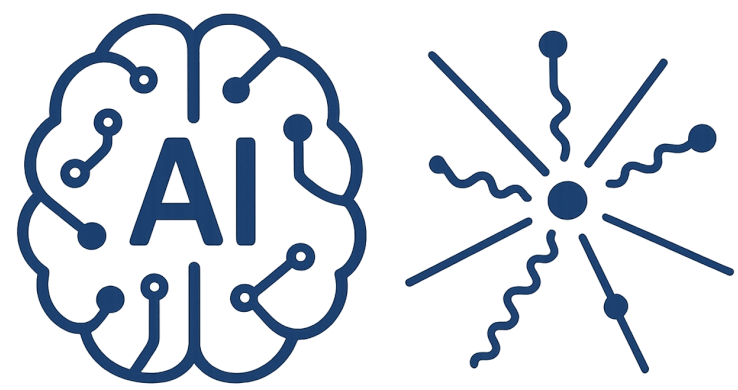
$$z = y + x_2$$

$$z = x_1 x_2 + x_2$$

$$\frac{\partial L}{\partial x_2} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x_2}$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial y}$$





Example

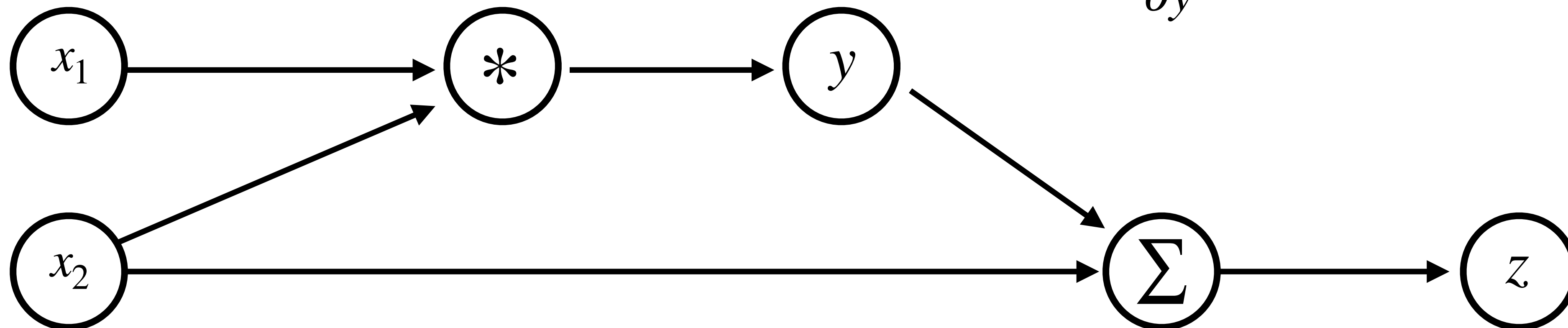
Backward Mode

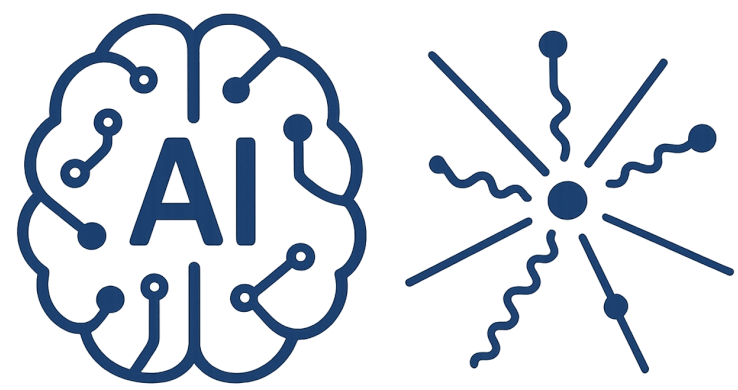
$$z = y + x_2$$

$$z = x_1 x_2 + x_2$$

$$\frac{\partial L}{\partial x_2} = dout \times 1$$

$$\frac{\partial L}{\partial y} = dout \times 1$$

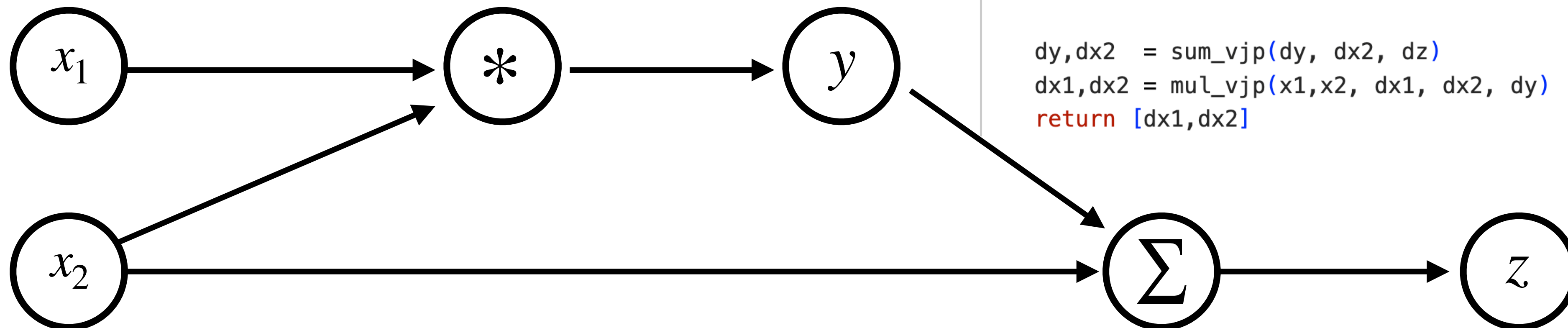




Example

Backward Mode

$$z = x_1 x_2 + x_2$$



```
def mul_vjp(x1,x2,dx1,dx2,dout):
```

```
    dx2 += dout * x1
```

```
    dx1 += dout * x2
```

```
    return dx1,dx2
```

```
def sum_vjp(dx1,dx2,dout):
```

```
    dx1 += dout * 1
```

```
    dx2 += dout * 1
```

```
    return dx1,dx2
```

```
def backward_vjp(x,dz):
```

```
    x1,x2 = x
```

```
    #zero gradients
```

```
    dy = 0
```

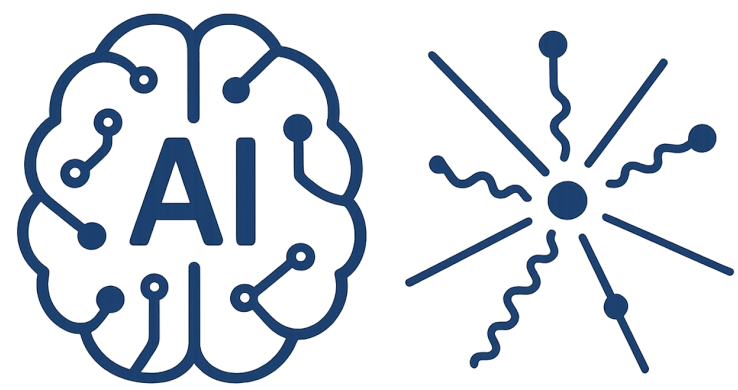
```
    dx1 = 0
```

```
    dx2 = 0
```

```
    dy,dx2 = sum_vjp(dy, dx2, dz)
```

```
    dx1,dx2 = mul_vjp(x1,x2, dx1, dx2, dy)
```

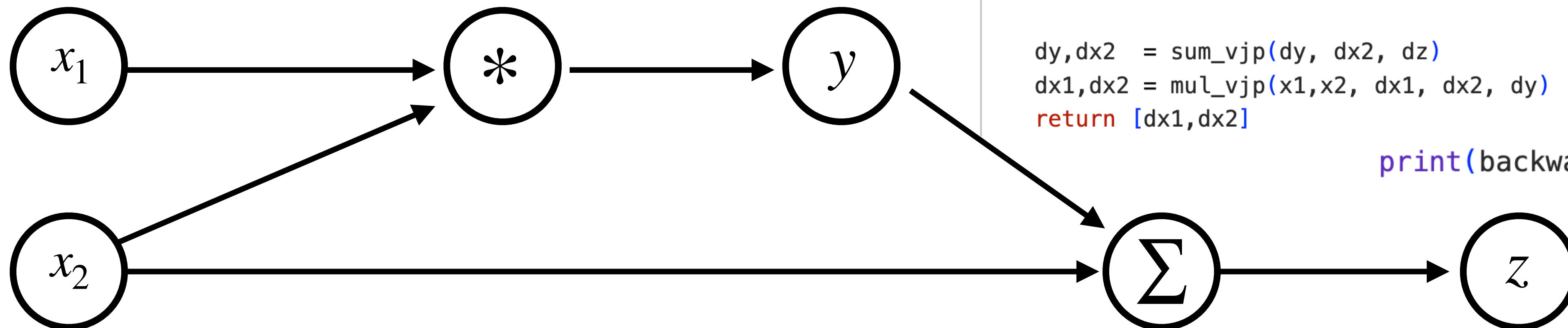
```
    return [dx1,dx2]
```



Example

Backward Mode

$$z = x_1 x_2 + x_2$$



```
def mul_vjp(x1,x2,dx1,dx2,dout):
```

```
    dx2 += dout * x1
```

```
    dx1 += dout * x2
```

```
    return dx1,dx2
```

```
def sum_vjp(dx1,dx2,dout):
```

```
    dx1 += dout * 1
```

```
    dx2 += dout * 1
```

```
    return dx1,dx2
```

```
def backward_vjp(x,dz):
```

```
    x1,x2 = x
```

```
    #zero gradients
```

```
    dy = 0
```

```
    dx1 = 0
```

```
    dx2 = 0
```

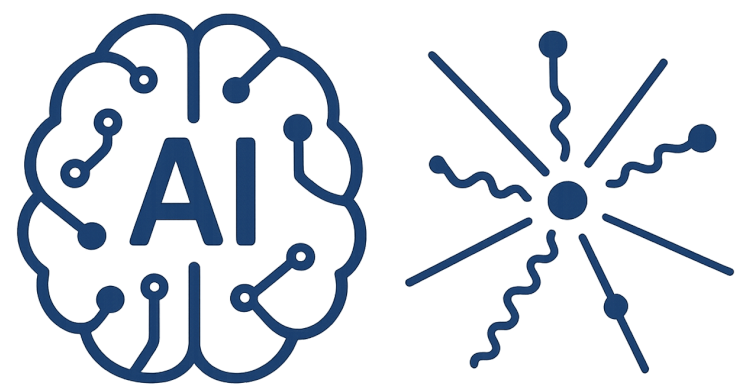
```
    dy,dx2 = sum_vjp(dy, dx2, dz)
```

```
    dx1,dx2 = mul_vjp(x1,x2, dx1, dx2, dy)
```

```
    return [dx1,dx2]
```

```
print(backward_vjp([2,4],1.0))
```

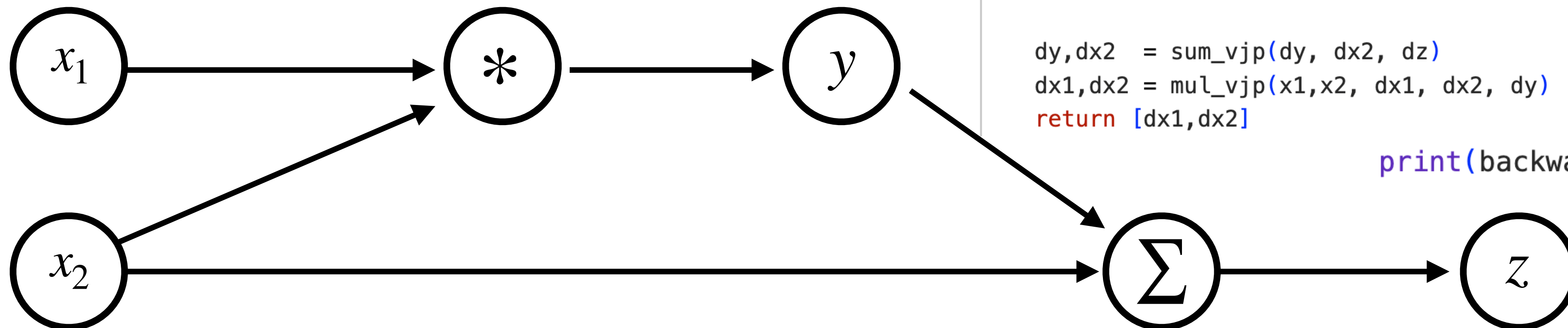
?



Example

Backward Mode

$$z = x_1 x_2 + x_2$$



```
def mul_vjp(x1,x2,dx1,dx2,dout):
```

```
    dx2 += dout * x1
```

```
    dx1 += dout * x2
```

```
    return dx1,dx2
```

```
def sum_vjp(dx1,dx2,dout):
```

```
    dx1 += dout * 1
```

```
    dx2 += dout * 1
```

```
    return dx1,dx2
```

```
def backward_vjp(x,dz):
```

```
    x1,x2 = x
```

```
    #zero gradients
```

```
    dy = 0
```

```
    dx1 = 0
```

```
    dx2 = 0
```

```
    dy,dx2 = sum_vjp(dy, dx2, dz)
```

```
    dx1,dx2 = mul_vjp(x1,x2, dx1, dx2, dy)
```

```
    return [dx1,dx2]
```

```
print(backward_vjp([2,4],1.0)) [4,3]
```